Portfolio

복잡한 기능을 사용자가 이해하고 편하게 쓸 수 있도록 만드는 프론트엔드 개발자, 민경빈입니다.

연락처
☎ +82 1057144138
✉ minwki32@naver.com
🎂 2001년 4월 19일

언어
🇰🇷 한국어
🇬🇧 영어

Education
평택 한광고등학교 졸업
2017년 2월~2020년 2월

숭실대학교 컴퓨터학부 학사 (졸업예정)
2020년 2월~2026년 8월

Portfolio
GITHUB : gyeongbibin (민경빈)
VELOG : https://velog.io/@minwki

Introduce

안녕하십니까!

AI로 개발 속도를 높이되, 설계와 검증은 직접 책임지는 프론트엔드 개발자 민경빈입니다.

AI가 만든 코드가 동작한다고 해서 곧바로 좋은 코드라고 생각하지는 않습니다. 먼저 해결할 문제와 지켜야 할 조건을 정하고, 결과가 사용자에게 필요한 동작과 팀의 개발 규칙을 충족하는지 확인합니다.

기능을 구현하기 전에는 왜 필요한지, 어디까지 구현할지, 실패하면 어떻게 처리할지를 정리합니다. 구현 후에는 상태 흐름과 렌더링, 접근성, 성능, 유지보수성을 점검합니다. 제가 이해하고 설명할 수 있는 코드를 만드는 것이 기준입니다.

요구사항을 명세로 작성하고, 테스트와 Storybook으로 동작을 확인해 왔습니다. CI와 성능 지표로 변경의 영향도 살핍니다. AI는 구현 방법을 탐색하고 반복 작업을 줄이는 데 활용하되, 어떤 방법을 선택할지와 결과를 받아들일지는 직접 판단합니다.

협업에서는 코드 작성 규칙과 리뷰 기준을 문서로 남기고, 반복해서 확인해야 할 항목은 자동 검사로 연결했습니다. Git 브랜치와 PR 규칙도 팀과 맞춰, 각자 빠르게 개발하더라도 다른 사람이 이해하고 이어서 작업할 수 있도록 했습니다.

앞으로도 AI를 어디에 활용하고 어떤 결과를 검증해야 하는지 판단하는 개발자가 되고자 합니다. 사용자가 AI의 결과를 이해하고 직접 수정할 수 있는 화면과 흐름을 만드는 데 관심이 있습니다.

감사합니다.

주요 프로젝트 및 대외 활동

일본 맛집 큐레이팅 및 예약 대행 서비스

HASHI - 발견부터 예약까지

2026년 6월~ 진행 중

1. 담당 업무

▾ 리뷰 작성부터 조회·삭제까지 구현

방문을 완료한 예약에서 리뷰를 작성하도록 하고, 이미지 업로드·목록·상세·삭제 API를 연결했습니다. 목록에는 커서 기반 무한 스크롤과 로딩·오류·빈 화면을 구현했습니다.
HDS의 Button·StarRating·Textarea를 사용해 입력 UI를 통일했습니다. 리뷰를 삭제하면 목록과 리뷰 개수, 예약 상태가 함께 갱신되도록 캐시를 처리했습니다.

▾ 지도에서 식당을 찾고 상세 정보를 확인하는 탐색 화면 구현

Google Maps SDK를 연동해 지역 요약 마커, 카테고리별 식당 핀, 상세 패널을 구현했습니다. 지도를 확대하거나 식당을 선택하면 해당 정보가 표시되도록 연결했습니다.
검색어·카테고리·정렬 조건과 '현 지도에서 검색' 기능을 조합했습니다. 상세 화면을 열었다가 목록으로 돌아와도 검색 조건과 스크롤 위치를 유지하도록 상태를 나눠 관리했습니다.
HDS DragPanel·Dialog로 드래그 가능한 목록과 상세 패널, 사진 뷰어를 만들었습니다. 모달을 닫으면 포커스를 복원하고, 열린 동안에는 배경 조작을 막았습니다. 지도 SDK 로딩 실패 시 안내와 재시도 기능도 구현했습니다.

▾ 배포 자동화·오류 알림·검색엔진 노출 설정

Vercel Preview·Production 배포, Vitest·Playwright 테스트, Sentry·Discord 오류 알림을 구성했습니다. PR마다 실제 Preview 페이지를 Lighthouse로 3회 측정하도록 연결했습니다.
동적 URL 생성은 generatePath로 통일했습니다. 공개 페이지 6종에는 빌드 시 HTML 생성, 메타데이터, JSON-LD, sitemap, robots 설정을 적용하고, 없는 주소는 HTTP 404로 응답하도록 했습니다.
Google Search Console과 네이버 서치어드바이저에 사이트맵을 등록해 크롤링·색인 상태를 확인할 수 있도록 했습니다.

2. 트러블 슈팅 및 성과

▾ 이미지 일부 업로드 실패 시 리뷰 등록 중단 및 재시도 처리

문제 상황
여러 이미지를 presigned URL로 올릴 때 일부 파일만 실패하면, 성공한 파일을 다시 전송하거나 이미지가 빠진 리뷰를 등록할 수 있었습니다.

기존 방식의 한계
파일별 업로드 결과와 리뷰 등록 요청이 분리되어 있지 않았습니다. 일부 업로드가 실패해도 리뷰 등록이 진행될 수 있었고, 전체를 재시도하면 성공한 파일까지 다시 전송했습니다.

해결 방법
백엔드와 협의해 이미지별 presigned URL을 한 번에 발급받도록 API를 변경했습니다. 성공한 파일은 유지하고 실패한 파일을 재시도하며, 모든 fileKey를 확보한 경우에만 리뷰 등록 API를 호출하도록 했습니다. 재시도 후에도 실패한 파일이 남으면 등록을 중단했습니다.

성과
성공한 이미지의 중복 전송을 막고, 선택한 이미지가 일부 빠진 상태로 리뷰가 등록되지 않도록 했습니다.

▾ SPA를 유지하면서 검색엔진이 읽을 수 있는 HTML 제공

문제 상황
React SPA에서는 JavaScript 실행 후에 식당·메뉴 정보와 메타데이터가 생성됐습니다. 모든 URL에 같은 SPA 문서를 반환해, 존재하지 않는 주소도 HTTP 200으로 응답했습니다.

기존 방식의 한계
브라우저에서 메타데이터만 바꿔서는 검색엔진이 공개 콘텐츠와 URL을 안정적으로 파악하기 어려웠습니다. 전체를 SSR 프레임워크로 옮기면 기존 라우팅·배포·PWA 구조의 변경 범위가 컸습니다.

해결 방법
기존 SPA는 유지하고, 프로덕션 빌드에서 공개 API 데이터를 받아 페이지 6종의 URL별 정적 HTML을 생성했습니다. 메타데이터·canonical·JSON-LD·sitemap·robots 설정과 없는 주소의 HTTP 404 응답도 함께 적용했습니다.

성과
JavaScript 실행 전에도 공개 페이지의 핵심 콘텐츠를 읽을 수 있도록 했습니다. Google Search Console과 네이버 서치어드바이저에 사이트맵을 제출하고 크롤링·색인 상태를 확인할 수 있게 했습니다.

▾ 빠른 페이지 이동 시 로딩 화면이 깜빡이는 문제 개선

문제 상황
페이지별 코드 스플리팅을 적용한 뒤, 페이지가 빠르게 준비돼도 Suspense의 로딩 화면이 잠깐 나타났다 사라졌습니다.

기존 방식의 한계
로딩 화면을 즉시 표시하면 짧은 전환에도 깜빡임이 생겼습니다. 반대로 모든 전환에 최소 대기 시간을 적용하면 이미 준비된 페이지까지 늦게 보여 주게 됐습니다.

해결 방법
Google RAIL 모델의 응답 시간 기준을 참고해 로딩 화면을 150ms 뒤에 표시하도록 했습니다. 그 전에 페이지가 준비되면 대기 없이 전환했습니다. 로딩 화면이 표시된 경우에만 최소 300ms 동안 유지해, 나타나자마자 사라지는 현상을 줄였습니다.

성과
빠른 전환은 로딩 화면이나 추가 지연 없이 처리했습니다. 로딩 화면이 필요한 전환에는 150ms의 표시 지연과 최소 300ms의 노출 시간을 적용해, 전환 시작부터 최소 약 450ms의 일정한 피드백을 제공했습니다. 실제 페이지가 더 늦게 준비되면 준비가 끝날 때까지 로딩을 유지합니다.

고민한 점

AI에게 맡길 작업과 개발자가 직접 판단할 일은 무엇인가

AI는 반복 구현과 코드 탐색을 빠르게 처리했지만, 팀의 의도를 놓치면 동작하는 코드에서도 모듈 간 역할 구분을 어기거나 오류 처리와 문서를 빠뜨렸습니다. 그래서 코드 작성은 AI의 도움을 받되, 사용자에게 보여 줄 상태와 구조 선택, 실패 시 재시도하거나 중단할 조건은 직접 결정했습니다. 생성한 코드의 양보다 선택한 이유를 설명할 수 있는지, 테스트와 문서로 결과를 확인했는지를 기준으로 삼았습니다.

성장 스토리

처음에는 AI로 코드 작성과 반복 작업을 줄이는 데 집중했습니다. 이후 프로젝트의 AI 작업 환경을 익히면서, 구현 전에 필요한 문서와 검증 기준을 먼저 전달하게 됐습니다.
AGENTS.md에서 작업별 문서를 찾고, docs/에 정리된 팀 규칙을 따랐습니다. 명세와 작업 계획에는 목표, 이번에 하지 않을 일, 실패 시 처리 방법을 적었습니다.
구현 후에는 .agents/의 작업 지침과 체크리스트로 API 요청·응답 형식, 쿼리 키, 캐시 갱신, 화면 상태, 문서 반영 여부를 확인했습니다. 리뷰 상태 정리, URL 생성 방식 통일, Lighthouse 검사, SEO 프리렌더링에 이 과정을 적용했습니다. AI가 만든 결과를 그대로 받아들이기보다 필요한 정보를 먼저 제공하고 결과를 직접 검증하는 방식으로 작업하게 됐습니다.

30일 현금흐름 분석 및 금융 회복 지원 서비스

RECOVERY-CLIENT

2026년 8월~ 2026년 9월

1. 담당 업무

▾ AI가 작업 규칙을 읽고 위험한 명령을 막는 개발 환경 구성

AGENTS.md에서 작업에 필요한 문서를 안내하도록 했습니다. 팀 규칙은 docs/, 체크리스트·작업 절차·검증 스크립트는 .agents/, AI 세션 설정과 명령 실행 검사는 .codex/에 나눠 정리했습니다.
SessionStart hook으로 작업 시작 시 프로젝트 규칙, 패키지 매니저, 브랜치와 검증 기준을 전달했습니다. PreToolUse hook에서는 셸 명령을 검사해, 지정하지 않은 패키지 매니저나 작업 내용을 지울 수 있는 Git·삭제 명령을 실행 전에 차단했습니다.

▾ FrontPrep을 실제 프로젝트에 적용해 설정 동작 검증

RECOVERY-CLIENT에 FrontPrep beta.2와 schema v2 형식의 변경 기록 파일(manifest)을 적용했습니다. CLI가 생성한 코드 품질 검사, 테스트, Git Hook, CI 설정이 실제 프로젝트에서도 동작하는지 확인했습니다.

2. 트러블 슈팅 및 성과

▾ 문서 누락과 위험한 명령 실행을 자동 검사로 방지

문제 상황
AI가 긴 프롬프트나 이전 대화에만 의존하면, 필요한 문서를 읽지 않거나 저장소 규칙과 다른 패키지·Git 명령을 실행할 수 있었습니다.

기존 방식의 한계
규칙을 글로만 전달하면 세션마다 지키는 정도가 달랐습니다. 문서가 빠졌는지, 위험한 명령이 실제로 차단되는지도 반복해서 확인하기 어려웠습니다.

해결 방법
AGENTS.md에서 작업별 필수 문서를 안내하고, 팀 규칙과 체크리스트·작업 절차·검증 스크립트를 분리해 정리했습니다. 명령 검사에서는 인용부호, 여러 명령의 조합, 셸을 통해 다시 실행하는 명령까지 분석해 차단 대상인지 확인했습니다.

성과
check:harness로 필수 경로 21개, 문서 연결, 필수 스크립트의 존재를 검사했습니다. 하네스 테스트 8개와 hook 테스트 12개로 잘못된 hook JSON, 문서 누락, 지정하지 않은 패키지 매니저, 파괴적인 Git 명령을 검사할 수 있도록 했습니다.

고민한 점

어떤 행동은 자동으로 막고, 어떤 판단은 개발자에게 맡길 것인가

명령을 지나치게 차단하면 정상적인 개발까지 불편해지고, 문서로만 안내하면 규칙을 놓칠 수 있습니다. 작업 내용을 지우거나 저장소 규칙을 명확히 어기는 명령은 hook으로 막았습니다. 반면 구현 방법처럼 상황에 따라 달라지는 선택은 개발자가 문서와 체크리스트를 참고해 판단하고, 검증 결과를 남기도록 했습니다. hook만으로 모든 위험을 막을 수 있다고 보지 않고, 문서와 테스트를 함께 사용했습니다.

프론트엔드 자동화 CLI FrontPrep

frontprep

2026년 8월~ 진행 중

1. 담당 업무

▾ 기존 프로젝트의 설정을 확인하고, 실패하면 되돌리는 CLI 구현

프로젝트 분석 → 변경 계획 생성 → 파일 수정 → 의존성 설치 → 검증 → 변경 기록 저장 순서로 실행 단계를 나눴습니다. 시작 전에는 Git에 저장하지 않은 변경이 없는지 확인했습니다.
계획을 만든 뒤 파일이 바뀌면 STALE_PLAN 오류로 중단하고, 적용 중 실패하면 수정한 파일·lockfile·manifest를 복원하도록 했습니다.
기존 설정과 충돌하거나 CLI가 수정해도 되는 파일인지 확인할 수 없으면 자동으로 합치지 않고 중단했습니다. check 명령은 파일을 바꾸지 않고 상태만 검사하도록 분리했습니다.

프론트엔드 자동화 CLI FrontPrep (계속)

1. 담당 업무 (계속)

▾ 지원 환경과 설정 파일 버전 변경 방식 명시

지원 환경을 Node 22.22.1+, Next.js 16 App Router, TypeScript 5, pnpm 10으로 명시했습니다.
독립 앱 또는 pnpm workspace에서 선택한 앱 하나를 설정하도록 했습니다. Pages Router, JavaScript 전용 프로젝트, 다른 패키지 매니저, workspace 전체 자동 설정은 v1에서 제외했습니다.
변경 기록 파일(manifest)을 schema v2로 개선하고, 앱 폴더와 저장소 최상위 폴더에 적용할 설정을 구분했습니다. 기존 manifest도 설정 적용 과정에서 변환하고, 실패하면 되돌릴 수 있도록 했습니다.

▾ 실제 배포 패키지 설치와 생성된 프로젝트 동작 검증

Quality·Tailwind·Test·Git Hooks·CI의 5개 설정 모듈을 제공했습니다. 같은 명령을 다시 실행해도 불필요한 변경이 생기지 않는지 확인했습니다.
저장소 소스만 테스트하지 않고, 실제 npm 배포용 압축 파일(tarball)을 독립된 환경에 설치했습니다. 설치된 CLI 명령과 생성된 프로젝트의 품질 검사·테스트·빌드를 검증했습니다.
@mingyeongbin/frontprep@0.1.0-beta.2와 schema v2 manifest를 RECOVERY-CLIENT에 적용해 실제 사용하는 프로젝트에서도 확인했습니다.

2. 트러블 슈팅 및 성과

▾ 프로젝트마다 다른 기존 설정을 보호하며 변경 적용

문제 상황
CLI를 설치하는 프로젝트마다 설정 파일, lockfile, Git 상태, workspace 구조가 달랐습니다. 파일을 일괄 생성하거나 덮어쓰면 기존 코드와 설정을 망가뜨릴 수 있었습니다.

기존 방식의 한계
변경할 내용을 정하지 않고 바로 수정하면, 중간에 실패했을 때 무엇을 되돌려야 하는지 알기 어려웠습니다. 기존 설정과 충돌할 때의 처리 기준도 필요했습니다.

해결 방법
분석 결과로 변경 계획을 만들고, 적용 직전에 파일이 달라졌으면 중단하도록 했습니다. 실패하면 파일·lockfile·manifest를 복원하고, CLI가 수정해도 되는지 확인되지 않은 파일은 자동으로 합치지 않았습니다.

성과
어떤 파일을 바꾸고, 언제 중단하며, 실패하면 무엇을 복원할지를 명확히 했습니다. 환경이 다른 프로젝트에서도 기존 설정을 확인한 뒤 변경을 적용하도록 했습니다.

▾ 소스 테스트를 넘어 사용자가 설치할 패키지까지 검증

문제 상황
로컬 소스는 정상이어도 이전 빌드 결과가 패키지에 포함되면, 사용자는 최신 수정이 빠진 CLI를 설치할 수 있었습니다.

기존 방식의 한계
소스 코드 테스트만으로는 배포 파일의 CLI 실행 경로(bin), 빌드 결과물, 설치 후 동작까지 확인할 수 없었습니다.

해결 방법
prepack 단계에서 반드시 빌드하도록 했습니다. 실제 npm 배포용 tarball을 독립된 환경에 설치하고, CLI 명령 실행과 생성된 프로젝트의 품질 검사·테스트·빌드를 확인했습니다.

성과
개발 중인 소스뿐 아니라 사용자가 내려받을 패키지를 검사하게 됐습니다. 버전·패키징 문제와 기존 환경에서의 호환성 문제를 배포 전에 확인할 수 있도록 했습니다.

고민한 점

지원 환경을 넓힐 것인가, 검증할 수 있는 범위에 집중할 것인가

여러 라우터와 패키지 매니저를 지원하면 더 많은 프로젝트에서 쓸 수 있지만, 검증하지 못한 환경의 문제는 사용하는 개발자가 떠안게 됩니다. v1은 Next.js 16 App Router·TypeScript·pnpm으로 범위를 제한했습니다. 대신 그 안에서 이전 manifest의 변환, workspace 내 앱 선택, 실패 시 복원이 제대로 동작하는지 확인하는 데 집중했습니다.

성장 스토리

프로젝트마다 반복하던 품질 검사와 협업 설정을 CLI로 만들었습니다. RECOVERY-CLIENT에 beta.2를 직접 적용하면서, 도구를 만드는 입장에서는 놓치기 쉬운 설치 환경과 기존 설정의 차이도 확인했습니다. 이후에는 기능을 늘리기 전에 지원 버전, 이전 설정과의 호환성, 의존성 변경, 실패 시 복구 방법을 먼저 정하게 됐습니다.

AI 대화로 프레젠테이션을 만드는 Snapdeck

1. 담당 업무

▾ AI 대화부터 슬라이드 확인·수정까지 제작 화면 구현

콘텐츠 생성 요청, AI와의 대화, 슬라이드 확인·수정으로 이어지는 화면을 구현했습니다. 채팅 내용과 슬라이드 데이터를 연결해 하나의 제작 흐름으로 사용할 수 있도록 했습니다.
화면 표시, 대화 상태, 비동기 응답 처리, 페이지 상태를 컴포넌트와 커스텀 훅으로 나눠 관리했습니다.

▾ 공통 UI와 시각 테스트 구성, 서버 데이터와 화면 데이터 분리

Figma의 디자인 기준을 토큰과 공통 컴포넌트로 만들었습니다. Storybook·Chromatic을 연결해 팀원이 같은 UI를 확인하고, 변경 전후의 화면을 비교할 수 있도록 했습니다.
TanStack Query로 서버 데이터를 관리하고, 서버 응답을 화면에 맞는 데이터로 변환하는 코드를 분리했습니다. API 형식이 바뀌어도 UI 컴포넌트의 수정 범위를 줄일 수 있도록 했습니다.

2. 트러블 슈팅 및 성과

▾ ChatSection에서 대화 상태와 AI 응답 처리를 분리

문제 상황
대화 이력, 프롬프트 입력, 펼침·접힘, AI 응답 처리와 화면 표시가 하나의 ChatSection에 모여 있었습니다.

기존 방식의 한계
동작 하나를 수정해도 전체 채팅 흐름에 영향이 없는지 확인해야 했습니다. AI 응답 방식이나 외부 데이터 연결을 바꿀 때 UI 코드까지 함께 수정해야 해 기능 확장과 테스트가 어려웠습니다.

해결 방법
대화 턴은 useContentChatTurns, 사용자 입력과 동작은 useContentChatSection, 비동기 응답은 Scheduler 훅에서 처리하도록 나눴습니다. ChatSection에는 컴포넌트를 조합하고 화면을 표시하는 코드만 남겼습니다.

성과
ChatSection을 229줄에서 110줄로 줄였습니다. 대화 상태와 응답 처리 방식을 UI 코드와 분리해 수정할 수 있게 했습니다.

▾ URL 검증을 분리해 다른 화면에서도 쓰는 TextField로 개선

문제 상황
공통 입력 컴포넌트가 Notion·Web URL 검증, 입력 유형 구분, 성공·오류 문구, 화면 표시까지 모두 처리해 다른 입력 화면에서 재사용하기 어려웠습니다.

기존 방식의 한계
URL 검증 조건이나 안내 문구를 바꿀 때 공통 컴포넌트도 수정해야 했습니다. 입력 유형이 늘어날수록 내부 조건 분기도 늘어났습니다.

해결 방법
URL 검증과 상태 판단은 부모 컴포넌트와 별도 유틸리티로 옮겼습니다. TextField는 전달받은 status와 helperText를 표시하고, 접근성 속성을 제공하도록 바꿨습니다.

성과
새 입력 유형을 추가할 때도 TextField 내부를 고치지 않고, 외부에서 검증 규칙과 안내 문구를 지정해 사용할 수 있게 했습니다.

▾ 슬라이드 순서 변경 후 화면과 서버 데이터를 일치시킴

문제 상황
슬라이드를 드래그해 순서를 바꾼 뒤, 화면에 보이는 순서와 서버에 저장된 순서가 달라질 수 있었습니다.

기존 방식의 한계
화면의 상태만 바꾸면 다시 조회하거나 다른 화면으로 이동했을 때 이전 순서가 나타날 수 있었습니다.

해결 방법
순서 변경 API를 호출하고, 서버가 반환한 결과를 TanStack Query 캐시에 반영했습니다. 이후 화면도 이 데이터를 기준으로 표시하도록 했습니다.

성과
순서를 바꾼 직후와 다시 조회한 화면이 같은 서버 데이터를 사용하도록 맞췄습니다.

고민한 점

화면·상태·데이터 변환 코드를 어디까지 나눌 것인가

채팅 로직을 한곳에 모으면 처음에는 흐름을 읽기 쉽지만, 상태나 응답 방식 하나를 바꿔도 UI 코드까지 수정해야 했습니다. 반대로 작은 훅으로 지나치게 나누면 실행 순서를 따라가기 어려웠습니다. 그래서 코드 길이보다 '함께 바뀌는 기능인가'를 기준으로 나눴습니다.
대화 턴은 useContentChatTurns, 입력과 가이드라인 동작은 useContentChatSection, 비동기 응답은 Scheduler 훅에 모았습니다. 서버 응답을 화면 데이터로 바꾸는 일은 Mapper가 맡고, TextField는 URL 검증 결과를 전달받아 표시하도록 했습니다. 각 코드가 무엇을 맡는지 설명할 수 있고, 다른 역할의 코드를 건드리지 않고 수정할 수 있는지를 확인했습니다.

성장 스토리

처음에는 Figma 화면과 인터랙션을 구현하는 데 집중했습니다. 기능이 늘고 API가 연결되자, 동작하는 화면을 만드는 것만으로는 변경에 대응하기 어렵다는 점을 알게 됐습니다.
ChatSection에 모인 대화 상태와 AI 응답 처리를 나눠 229줄을 110줄로 줄였고, URL 검증에 묶여 있던 TextFieldNotionModal을 여러 입력 화면에서 쓸 수 있는 TextField로 바꿨습니다. 서버 응답과 화면 데이터 사이에는 Mapper를 두어 API 변경을 처리할 위치도 명확히 했습니다.
Figma 기반 디자인 토큰과 Storybook·Chromatic을 구성하면서는 제가 맡은 화면뿐 아니라 팀원이 공통 UI를 개발하고 검증하는 방법도 함께 고민했습니다. 이후에는 기능을 추가하기 전에 변경될 부분과 공통으로 유지할 부분을 먼저 구분하게 됐습니다.

대학생 연합 IT 벤처 창업 동아리 SOPT 37기 WEB 파트

2025년 9월~2026년 1월

1. 활동 요약

합동세미나

티켓베이 리디자인 프로젝트에서 기획·디자인 파트와 처음 협업했습니다. 기획서와 디자인을 구현하면서 사용자 흐름과 기술적으로 가능한 범위를 함께 검토했습니다. 디자인 의도를 확인하고, 구현이 어려운 부분에는 대안을 제안하는 소통 방식을 배웠습니다.

솝커톤

하루 안에 서비스를 기획하고 개발·배포했습니다. 모든 기능을 넣기보다 핵심 기능을 먼저 정하고, 역할을 나눠 완성하는 데 집중했습니다.
짧은 일정에서는 무엇을 구현할지뿐 아니라 무엇을 제외할지도 빨리 결정해야 했습니다. 이후 프로젝트에서도 일정과 인력을 고려해 우선순위를 정하는 기준이 됐습니다.

웹 세미나 및 스터디

프론트엔드 기초 세미나와 매주 과제를 통해 부족했던 내용을 정리했습니다. 서로 코드리뷰를 주고받으며, 기능이 동작하는지뿐 아니라 구현한 이유를 설명할 수 있는지도 확인했습니다.
호기심스 스터디에서는 번들러와 빌드 도구를 살펴보며, 개발 환경이 구성되고 코드가 배포 파일로 만들어지는 과정을 학습했습니다.

고민한 점

수정 요청만 하는 리뷰에서, 선택한 이유를 묻는 리뷰로

이전에는 충돌 없이 기능이 동작하면 별도 논의 없이 머지하곤 했습니다. SOPT에서 과제와 스터디의 코드리뷰를 주고받으며, 같은 기능도 여러 방식으로 구현할 수 있고 선택한 이유를 이해하는 과정이 필요하다는 것을 배웠습니다.
처음에는 다른 사람의 코드에 의견을 남기는 것이 부담스러웠습니다. 이후에는 '이렇게 바꾸세요'보다 '왜 이 구조를 선택했나요?'처럼 의도를 묻고 대안을 함께 비교하려고 했습니다. 코드리뷰를 정답을 판정하는 일보다, 서로의 판단 기준을 맞추고 더 나은 선택을 찾는 대화로 받아들이게 됐습니다.

성장 스토리

개인 프로젝트에서는 맡은 기능이 동작하는 데 집중했다면, SOPT에서는 기획·디자인·다른 개발자와 구현 범위와 이유를 공유하는 경험을 쌓았습니다. 코드리뷰에서 제 선택을 설명하고 질문을 받으면서, 제 코드를 다시 읽고 개선점을 찾는 습관도 생겼습니다.
세미나, 스터디, 합동세미나, 솝커톤을 병행하며 일정과 우선순위를 조율했습니다. 시간이 부족할 때는 기능 수를 늘리기보다 핵심 흐름을 완성하는 쪽을 선택했습니다.
동작하는 코드도 다시 점검했습니다. 불필요한 로직과 렌더링 흐름을 살펴보고, 이미지 로딩을 개선하기 위해 WebP 적용도 시도했습니다. 이 경험을 바탕으로 이후 프로젝트에서는 구현한 이유를 설명하고, 함께 작업할 사람이 이해하기 쉬운 코드를 만들려고 노력하고 있습니다.

SIDE PROJECT

활동 요약

숭실 IT 팀 프로젝트 (장려상 수상) (2025.05~2025.11)
Heal-pick (신장질환 환자들을 위한 건강맞춤 서비스)
Github : HeathyPick_FE

IT 벤처 창업 동아리 SOPT (2025.09 ~ 2026.08)
TicketBay (티켓베이 리디자인) / Github : 37-COLLABORATION-WEB-TICKETBAY
바스락 (SNS 밈 공유 서비스) / Github : 37-SOPKATHON-WEB-WEB1
Snapdeck (PPT AI Agent 리디자인) / Github : 38-COLLABORATION-WEB-SNAPDECK
HASHI (일본 맛집 큐레이션 및 예약 서비스) / Gihhub : HASHI-CLIENT

숭실대학교 중앙축구 동아리 SSC (2023.02 ~ 2025.12)
SSC 중앙 축구 동아리 50기 회장 수행 (2024.02~2024.12)
SUFA 리그 무패우승(2025)